Protobuf Dynamic Transcoder — PRD

Overview

A .NET library that decodes protobuf-encoded bytes into a structured internal model and serializes that model to JSON, using only raw `.proto` schema files as input. No code generation, no compiled types, no `protoc` toolchain required.

Problem

The official C# protobuf library (`Google.Protobuf`) has no `DynamicMessage` . Java and C++ have had this for over a decade. The C# library requires generated types at compile time — there is no way to decode a protobuf message at runtime given only the schema.

This is a known, long-standing gap:

- Issue #658: Add DynamicMessage support for C# — opened **July 2015**, still open
- Issue #10422: Support for DynamicMessage in C# — confirms no support, no planned timeline

`protobuf-net` also explicitly does not support schema-to-deserializer at runtime.

Meanwhile, Java's `DynamicMessage` (source) paired with `JsonFormat` (source) does exactly what we need. We port that architecture to C#.

Solution

Port the Java protobuf library's dynamic message architecture to C#, from scratch, targeting modern .NET APIs and full AOT compatibility. The library consists of:

1. A `.proto` file parser
2. A resolved descriptor model (type registry with cross-references)
3. A wire format reader (protobuf binary decoding)
4. A dynamic message model (runtime representation of decoded messages)
5. A JSON serializer (proto3 canonical JSON mapping)

Use Cases

- Decode and inspect protobuf messages in observability/debugging tools
- Transcode protobuf → JSON in gateways and middleware
- Schema-driven protobuf message inspection in test tooling
- Any scenario where the message types are not known at compile time

Constraints

- **Zero external dependencies.** No `Google.Protobuf`, no `protobuf-net`. Only `System.Text.Json` (ships with .NET runtime).
- **AOT compatible.** No `System.Reflection`, no `Emit`, no `Type.GetType()`. Must work with `PublishAot` on .NET 10.
- **Decode only.** See "Not Building" below.

Stages

Stage 1: MVP — Proto3 Decode

Decode proto3 messages given `.proto` schema files. Produce an internal `DynamicMessage` model. Serialize to JSON.

Covers: all proto3 scalar types, nested messages, repeated fields, maps, oneofs, enums, well-known types (Timestamp, Duration, Struct, wrappers, Any, FieldMask, Empty).

Spec: `protobuf-transcoder-mvp-spec.md`

Stage 2: Proto2 + Editions 2024

Add full proto2 support (explicit field presence, default values, required fields, groups, extensions) and Editions 2024 (feature resolution that maps to proto2/proto3 behaviors). After this stage the library handles the entire protobuf ecosystem.

Spec: `protobuf-transcoder-stage2-spec.md`

Stage 3: Performance Hardening (tentative)

Object pooling for `DynamicMessage` instances, `FrozenDictionary` for descriptor lookups, SIMD varint decoding, benchmarking suite. Scope TBD based on profiling after Stage 2.

Not Building

Feature	Rationale
JSON → Protobuf encoding	Not a requirement. The use cases are decode/inspect/transcode-out. If this ever becomes needed it can be layered on top of `DynamicMessage` + a `CodedWriter`, but the library is not designed around bidirectional transcoding.
gRPC integration / HTTP transcoding	Out of scope. This is a codec library, not a framework.
Service descriptors	Parsed by the proto parser for completeness, but not modeled or exposed beyond the raw parse output.
Streaming / multi-message decode	Protobuf wire format is not self-delimiting. Callers who need to decode concatenated length-delimited messages can handle framing themselves and call decode per message.
Source code info / comments	Not preserved in the descriptor model.

Design Decisions

DynamicMessage: mutable or immutable?

Java's `DynamicMessage` is immutable once built. It uses a mutable `Builder` for construction (including during parsing — `Builder.mergeFrom()` reads from the wire and then `build()` freezes it). See DynamicMessage.java and its inner `Builder` class.

We follow the same pattern: mutable `Builder` used internally during decode, immutable `DynamicMessage` returned to the caller. This is useful for testing (construct messages programmatically via the builder) and matches the proven Java design.

Extension auto-discovery

Extensions are auto-discovered from all `.proto` files loaded into the `DescriptorPool`. If

you loaded the file, its extensions are available. No explicit registration step. Unknown extensions encountered during decode are silently skipped (standard protobuf behavior). This is not needed for the MVP (proto3 has no extensions) but the descriptor model should not make it impossible — it just won't be wired up until Stage 2.

Success Criteria

- Round-trip correctness: decode a message encoded by `protoc` / Java / Go, produce JSON that matches the canonical proto3 JSON output from those implementations.
- Handles all `.proto` files in the `google/protobuf` well-known types without parse errors.
- Handles proto2, proto3, and editions 2024 `.proto` files (after Stage 2).
- Zero trimmer warnings under `PublishAot`.
- No external NuGet dependencies.

Dependencies & Risks

Risk	Mitigation
Proto parser complexity — the `.proto` grammar has edge cases (nested extends, group syntax, option value literals)	Implement incrementally: proto3 first (simpler), proto2 additions second. Use protobuf's own conformance test protos as validation.
Editions 2024 feature resolution is not widely documented outside Google's codebase	The feature set is small and well-defined in `descriptor.proto`. Editions desugar into proto2/proto3 behaviors — if those are correct, editions is config mapping.
AOT compatibility of `System.Text.Json` `Utf8JsonWriter` / `Utf8JsonReader`	These APIs are fully AOT-safe. No risk here.
Extensions can appear in any message from any file	The extension registry is scoped to the `DescriptorPool` (all loaded files). Extensions are just fields with a wider scope — the decode loop adds one extra lookup.

References

Official Protocol Buffers Documentation

- Wire Format Encoding Reference — tag format, varint encoding, wire types, length-delimited fields, groups
- Proto3 Language Guide — proto3 grammar, scalar types, maps, oneofs, options, JSON mapping
- Proto2 Language Guide — proto2 grammar, required/optional, extensions, groups, default values
- Editions Language Guide — edition syntax, features blocks, migration from proto2/proto3
- Proto3 JSON Mapping (ProtoJSON) — canonical JSON encoding spec, well-known type representations
- Techniques (self-describing messages) — DynamicMessage usage patterns, FileDescriptorSet

Proto Schema Definition

- `descriptor.proto` — the protobuf meta-schema defining `FileDescriptorProto`, `DescriptorProto`, `FieldDescriptorProto`, `FeatureSet`, etc.

C# DynamicMessage Gap (proof the problem exists)

- Issue #658: Add DynamicMessage support for C# — opened 2015, still open
- Issue #10422: Support for DynamicMessage in C# — confirms no support
- protobuf-net Issue #902: Deserialize with runtime schema — Marc Gravell confirms not implemented

Java Reference Implementation (what we're porting)

- `DynamicMessage.java` — the dynamic message implementation
- `Descriptors.java` — `FileDescriptor`, `Descriptor`, `FieldDescriptor`, `EnumDescriptor`, cross-reference resolution
- `CodedInputStream.java` — wire format reader (varint, fixed, length-delimited, groups)
- `JsonFormat.java` — JSON printer and parser, well-known type handlers

- `MessageReflection.java` — the actual `mergeFrom` decode loop that reads tags and dispatches to field descriptors
- `AbstractMessage.java` — base class for `DynamicMessage`, contains `mergeFrom(CodedInputStream)` wiring
- `ExtensionRegistry.java` — extension lookup by message type + field number
- `TypeRegistry.java` — type registry for resolving `Any` type URLs
- `UnknownFieldSet.java` — unknown field preservation during decode

C++ Reference (supplementary — clearer decode loop)

- `wire_format.cc` — `ParseAndMergePartial()` is the clearest implementation of the tag→field→dispatch decode loop
- `dynamic_message.h` — C++ `DynamicMessageFactory`